Week 4 Structured Notes: Basic Recursive Problems

1. Week 4 Main Goal

The goal of Week 4 is to start using recursion to solve real beginner-level problems.

In Weeks 1 to 3, you learned the background ideas:

- Week 1 taught what recursion is.
- Week 2 taught how recursion uses stack memory.
- Week 3 taught the main types of recursion.

Week 4 now asks you to apply those ideas to simple mathematical problems.

By the end of Week 4, you should be able to write and trace recursive functions for:

- Sum of first `n` natural numbers
- Factorial of a number
- Negative input handling for factorial
- Basic power function
- Optimized power function, also called fast power
- Recursive vs loop comparison
- Basic time and space complexity

The most important Week 4 idea is:

> A recursive problem can often be solved by reducing a big problem into a smaller version of the same problem.

Example:

```
sum(5) = sum(4) + 5
factorial(5) = factorial(4) * 5
power(2, 5) = power(2, 4) * 2
```

In each case, the function solves a smaller problem first, then uses that answer to solve the bigger problem.

---

2. Quick Revision Before Week 4

Before learning Week 4, remember these recursion basics.

2.1 Recursion

Recursion means a function calls itself.

Example:

```
void fun(int n) {
    if (n > 0) {
        fun(n - 1);
    }
}
```

Here, `fun()` calls itself, so it is recursive.

2.2 Base Case

The base case is the condition that stops recursion.

Without a base case, recursion can continue forever and may crash the program.

Example:

```
if (n == 0) {
    return 0;
}
```

This tells the function when to stop.

2.3 Recursive Case

The recursive case is the part where the function calls itself again.

Example:

```
return sum(n - 1) + n;
```

Here, `sum(n - 1)` is the recursive call.

2.4 Calling Phase

The calling phase happens when recursion keeps going deeper.

Example:

```
sum(5)
sum(4)
sum(3)
sum(2)
sum(1)
sum(0)
```

The function calls are moving toward the base case.

2.5 Returning Phase

The returning phase happens after the base case is reached.

Then each function call finishes and sends an answer back to the previous call.

Example:

```
sum(0) returns 0
sum(1) returns 1
sum(2) returns 3
sum(3) returns 6
sum(4) returns 10
sum(5) returns 15
```

Many mathematical recursive functions do their main calculation during the returning phase.

---

3. Why Week 4 Is Important

Week 4 is important because this is where recursion becomes useful for problem solving.

Before Week 4, you mainly studied how recursion behaves.

Now you learn how to use recursion to solve problems.

The main skill is to convert a mathematical definition into code.

For example:

```
sum(n) = sum(n - 1) + n
```

becomes:

```
return sum(n - 1) + n;
```

This is the bridge between mathematics and programming.

Simple Week 4 question:

```
Can I express this problem using a smaller version of the same problem?
```

If yes, recursion may be possible.

---

4. Problem 1: Sum of First N Natural Numbers

4.1 Meaning

The sum of first `n` natural numbers means adding numbers from `1` to `n`.

Example:

```
sum(5) = 1 + 2 + 3 + 4 + 5
sum(5) = 15
```

So the answer is `15`.

4.2 Recursive Thinking

Instead of adding all numbers directly, recursion thinks like this:

```
sum(5) = sum(4) + 5
sum(4) = sum(3) + 4
sum(3) = sum(2) + 3
sum(2) = sum(1) + 2
```

```
sum(1) = sum(0) + 1
sum(0) = 0
```

The problem keeps becoming smaller.

The original problem is:

```
Find sum(5)
```

The smaller problem is:

```
Find sum(4)
```

Then even smaller:

```
Find sum(3)
```

This continues until `sum(0)`.

4.3 Recursive Formula

```
sum(n) = sum(n - 1) + n
sum(0) = 0
```

This formula has two parts.

Base Case

```
sum(0) = 0
```

This stops the recursion.

Recursive Case

```
sum(n) = sum(n - 1) + n
```

This reduces the problem.

4.4 Code

```c
#include <stdio.h>

int sum(int n) {
    if (n == 0) {
        return 0;
    }

    return sum(n - 1) + n;
}

int main() {
    printf("%d\n", sum(5));
    return 0;
}
```

4.5 Output

```
15
```

4.6 Dry Run of `sum(5)`

Function call:

```
sum(5)
```

Calling Phase

```
sum(5) waits for sum(4) + 5
sum(4) waits for sum(3) + 4
sum(3) waits for sum(2) + 3
sum(2) waits for sum(1) + 2
sum(1) waits for sum(0) + 1
sum(0) returns 0
```

The function goes down until it reaches the base case.

Returning Phase

```
sum(1) = 0 + 1 = 1
sum(2) = 1 + 2 = 3
sum(3) = 3 + 3 = 6
sum(4) = 6 + 4 = 10
sum(5) = 10 + 5 = 15
```

Final answer:

```
15
```

4.7 Why Addition Happens During Returning Phase

Look at this line:

```
return sum(n - 1) + n;
```

The function cannot add `n` immediately because it first needs the answer of:

```
sum(n - 1)
```

So the function keeps waiting.

Only after the smaller call returns an answer can the addition happen.

That is why the main addition happens during the returning phase.

4.8 Stack View for `sum(5)`

At the deepest point, the stack looks like this:

```
sum(0)
sum(1)
sum(2)
sum(3)
sum(4)
```

```
sum(5)
main()
```

Each call has its own value of `n`.

That means:

```
sum(5) has n = 5
sum(4) has n = 4
sum(3) has n = 3
sum(2) has n = 2
sum(1) has n = 1
sum(0) has n = 0
```

These values do not overwrite each other because each call has its own stack frame.

4.9 Time and Space Complexity of Recursive Sum

Time Complexity

```
O(n)
```

Why?

For `sum(5)`, these calls are made:

```
sum(5)
sum(4)
sum(3)
sum(2)
sum(1)
sum(0)
```

That is about `n + 1` calls.

So the time complexity is `O(n)`.

Space Complexity

```
O(n)
```

Why?

Each recursive call creates a stack frame.

For input `n`, there are about `n + 1` stack frames.

So the space complexity is `O(n)`.

4.10 Loop Version of Sum

```
int sumLoop(int n) {
    int total = 0;

    for (int i = 1; i <= n; i++) {
        total = total + i;
    }

    return total;
}
```

4.11 Recursive Sum vs Loop Sum

Version	Time Complexity	Space Complexity	Reason
Recursive sum	`O(n)`	`O(n)`	Creates stack frames
Loop sum	`O(n)`	`O(1)`	Reuses the same variables

Both versions give the same answer.

But the loop usually uses less memory.

---

5. Problem 2: Factorial

5.1 Meaning

Factorial means multiplying a number by all positive numbers below it.

Example:

```
factorial(5) = 5 * 4 * 3 * 2 * 1
factorial(5) = 120
```

Factorial is written using `!`.

```
5! = 120
```

5.2 Recursive Thinking

Recursion thinks about factorial like this:

```
factorial(5) = factorial(4) * 5
factorial(4) = factorial(3) * 4
factorial(3) = factorial(2) * 3
factorial(2) = factorial(1) * 2
factorial(1) = factorial(0) * 1
factorial(0) = 1
```

Again, the problem keeps getting smaller.

5.3 Recursive Formula

```
factorial(n) = factorial(n - 1) * n
factorial(0) = 1
```

Base Case

```
factorial(0) = 1
```

Recursive Case

```
factorial(n) = factorial(n - 1) * n
```

5.4 Why Is `factorial(0)` Equal to `1`?

This is a common beginner question.

The simple explanation is:

```
factorial(1) = 1
```

But using the recursive formula:

```
factorial(1) = factorial(0) * 1
```

For this to be true:

```
factorial(0) * 1 = 1
```

So:

```
factorial(0) = 1
```

That is why `0! = 1`.

Another simple way to remember it:

> In multiplication, `1` is the safe stopping value.

If factorial stopped at `0`, the whole multiplication would become `0`, which would be wrong.

5.5 Code

```c
#include <stdio.h>

int factorial(int n) {
    if (n < 0) {
        return -1; // invalid input marker
    }

    if (n == 0) {
        return 1;
```

```c
    }

    return factorial(n - 1) * n;
}

int main() {
    printf("%d\n", factorial(5));
    return 0;
}
```

5.6 Output

```
120
```

5.7 Dry Run of `factorial(5)`

Function call:

```
factorial(5)
```

Calling Phase

```
factorial(5) waits for factorial(4) * 5
factorial(4) waits for factorial(3) * 4
factorial(3) waits for factorial(2) * 3
factorial(2) waits for factorial(1) * 2
factorial(1) waits for factorial(0) * 1
factorial(0) returns 1
```

Returning Phase

```
factorial(1) = 1 * 1 = 1
factorial(2) = 1 * 2 = 2
factorial(3) = 2 * 3 = 6
factorial(4) = 6 * 4 = 24
factorial(5) = 24 * 5 = 120
```

Final answer:

```
120
```

5.8 Important Point

Like recursive sum, factorial also does its main multiplication during the returning phase.

This line waits for a smaller answer first:

```
return factorial(n - 1) * n;
```

The function cannot multiply by `n` until `factorial(n - 1)` returns.

5.9 Time and Space Complexity of Recursive Factorial

Time Complexity

```
O(n)
```

Why?

The function reduces `n` by `1` each time.

Example:

```
factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)
factorial(0)
```

That is about `n + 1` calls.

Space Complexity

```
O(n)
```

Why?

Each call stays on the stack until the base case is reached.

So recursion uses stack memory proportional to `n`.

---

6. Negative Input Handling in Factorial

6.1 Why Negative Input Is a Problem

In this beginner version, factorial is only for `0` and positive integers.

These are valid:

```
factorial(0)
factorial(1)
factorial(5)
```

This is invalid:

```
factorial(-5)
```

6.2 What Goes Wrong Without Validation

Suppose you write this version:

```
int factorial(int n) {
    if (n == 0) {
        return 1;
    }

    return factorial(n - 1) * n;
}
```

Now call:

```
factorial(-1);
```

The calls become:

```
factorial(-1)
factorial(-2)
factorial(-3)
factorial(-4)
...
```

The value moves farther away from `0`.

So the base case is never reached.

This can cause stack overflow.

6.3 Correct Beginner-Safe Version

```
int factorial(int n) {
    if (n < 0) {
        return -1;
    }

    if (n == 0) {
        return 1;
    }

    return factorial(n - 1) * n;
}
```

Here:

```
if (n < 0) {
    return -1;
}
```

means:

```
Negative input is invalid.
Stop immediately.
```

6.4 Why Return `-1`?

`-1` is used as an invalid input marker.

Since factorial answers are normally positive for valid inputs, `-1` clearly means something went wrong.

Example:

```
factorial(5) = 120
factorial(0) = 1
factorial(-5) = -1
```

Here, `-1` means invalid input.

6.5 Better User-Friendly Handling

Instead of only printing `-1`, a full program can show a message.

```c
int result = factorial(n);

if (result == -1) {
    printf("Invalid input. Factorial is not defined for negative numbers.\n");
} else {
    printf("Factorial = %d\n", result);
}
```

This is clearer for the user.

---

7. Loop Version of Factorial

7.1 Code

```c
int factorialLoop(int n) {
    if (n < 0) {
        return -1;
    }

    int result = 1;

    for (int i = 1; i <= n; i++) {
```

```
        result = result * i;
    }

    return result;
}
```

7.2 Recursive Factorial vs Loop Factorial

Version	Time Complexity	Space Complexity	Reason
Recursive factorial	`O(n)`	`O(n)`	Creates stack frames
Loop factorial	`O(n)`	`O(1)`	Reuses one function frame

The recursive version is useful for learning recursion.

The loop version is usually better for memory.

---

8. Problem 3: Basic Power Function

8.1 Meaning

Power means repeated multiplication.

Example:

```
2^5 = 2 * 2 * 2 * 2 * 2
2^5 = 32
```

In function form:

```
power(2, 5) = 32
```

Here:

```
m = 2
n = 5
```

`m` is the base.

`n` is the exponent.

8.2 Recursive Thinking

The power function can be written recursively like this:

```
power(2, 5) = power(2, 4) * 2
power(2, 4) = power(2, 3) * 2
power(2, 3) = power(2, 2) * 2
power(2, 2) = power(2, 1) * 2
power(2, 1) = power(2, 0) * 2
power(2, 0) = 1
```

Again, the exponent becomes smaller every time.

8.3 Recursive Formula

```
power(m, n) = power(m, n - 1) * m
power(m, 0) = 1
```

Base Case

```
power(m, 0) = 1
```

Recursive Case

```
power(m, n) = power(m, n - 1) * m
```

8.4 Why Is `power(m, 0)` Equal to `1`?

Any non-zero number to the power of `0` is `1`.

Examples:

```
2^0 = 1
5^0 = 1
10^0 = 1
```

So the base case is:

```
if (n == 0) {
    return 1;
}
```

This stops the recursion.

8.5 Code

```
#include <stdio.h>

int power(int m, int n) {
    if (n == 0) {
        return 1;
    }

    return power(m, n - 1) * m;
}

int main() {
    printf("%d\n", power(2, 5));
    return 0;
}
```

8.6 Output

```
32
```

8.7 Dry Run of `power(2, 5)`

Function call:

```
power(2, 5)
```

Calling Phase

```
power(2, 5) waits for power(2, 4) * 2
power(2, 4) waits for power(2, 3) * 2
power(2, 3) waits for power(2, 2) * 2
power(2, 2) waits for power(2, 1) * 2
power(2, 1) waits for power(2, 0) * 2
power(2, 0) returns 1
```

Returning Phase

```
power(2, 1) = 1 * 2 = 2
power(2, 2) = 2 * 2 = 4
power(2, 3) = 4 * 2 = 8
power(2, 4) = 8 * 2 = 16
power(2, 5) = 16 * 2 = 32
```

Final answer:

```
32
```

8.8 Time and Space Complexity of Basic Power

Time Complexity

```
O(n)
```

Why?

The exponent decreases by `1` each time.

For example:

```
power(2, 5)
power(2, 4)
power(2, 3)
power(2, 2)
```

```
power(2, 1)
power(2, 0)
```

That is about `n + 1` calls.

Space Complexity

```
O(n)
```

Why?

All recursive calls wait on the stack until the base case returns.

---

9. Loop Version of Basic Power

9.1 Code

```c
int powerLoop(int m, int n) {
    int result = 1;

    for (int i = 1; i <= n; i++) {
        result = result * m;
    }

    return result;
}
```

9.2 Recursive Power vs Loop Power

Version	Time Complexity	Space Complexity	Reason
Recursive power	`O(n)`	`O(n)`	Creates recursive stack frames
Loop power	`O(n)`	`O(1)`	Reuses the same variables

Both calculate the same result.

The loop version uses less stack memory.

---

10. Problem 4: Optimized Power / Fast Power

10.1 Why Basic Power Is Slow

Basic power reduces the exponent by `1` each time.

Example:

```
power(2, 9)
```

Basic power goes like this:

```
9 -> 8 -> 7 -> 6 -> 5 -> 4 -> 3 -> 2 -> 1 -> 0
```

That is many calls.

For small inputs, this is fine.

For large inputs, it becomes inefficient.

Example:

```
power(2, 1000000)
```

This would make about one million recursive calls.

That is too much.

10.2 Main Idea of Fast Power

Fast power reduces the exponent by half instead of reducing it by one.

This makes the function much faster.

Example:

```
2^8
```

Basic thinking:

```
2^8 = 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2
```

Fast thinking:

```
2^8 = (2 * 2)^4
2^8 = 4^4
2^8 = (4 * 4)^2
2^8 = 16^2
2^8 = (16 * 16)^1
2^8 = 256
```

The exponent becomes smaller very quickly:

```
8 -> 4 -> 2 -> 1 -> 0
```

10.3 Rule for Even Exponent

If `n` is even:

```
m^n = (m * m)^(n / 2)
```

Example:

```
2^8 = (2 * 2)^4
2^8 = 4^4
```

So instead of reducing `8` to `7`, we reduce it to `4`.

10.4 Rule for Odd Exponent

If `n` is odd:

```
m^n = m * (m * m)^((n - 1) / 2)
```

Example:

```
2^9 = 2 * (2 * 2)^4
2^9 = 2 * 4^4
```

We keep one extra `m`, then reduce the remaining exponent.

10.5 Fast Power Code

```c
#include <stdio.h>

int fastPower(int m, int n) {
    if (n == 0) {
        return 1;
    }

    if (n % 2 == 0) {
        return fastPower(m * m, n / 2);
    }

    return m * fastPower(m * m, (n - 1) / 2);
}

int main() {
    printf("%d\n", fastPower(2, 9));
    return 0;
}
```

10.6 Output

```
512
```

10.7 Dry Run of `fastPower(2, 9)`

Function call:

```
fastPower(2, 9)
```

Since `9` is odd:

```
fastPower(2, 9) = 2 * fastPower(4, 4)
```

Now:

```
fastPower(4, 4)
```

Since `4` is even:

```
fastPower(4, 4) = fastPower(16, 2)
```

Now:

```
fastPower(16, 2)
```

Since `2` is even:

```
fastPower(16, 2) = fastPower(256, 1)
```

Now:

```
fastPower(256, 1)
```

Since `1` is odd:

```
fastPower(256, 1) = 256 * fastPower(65536, 0)
```

Now:

```
fastPower(65536, 0) = 1
```

Returning phase:

```
fastPower(256, 1) = 256 * 1 = 256
fastPower(16, 2) = 256
fastPower(4, 4) = 256
fastPower(2, 9) = 2 * 256 = 512
```

Final answer:

```
512
```

10.8 Why Fast Power Is Faster

Basic power for `2^9`:

```
9 -> 8 -> 7 -> 6 -> 5 -> 4 -> 3 -> 2 -> 1 -> 0
```

Fast power for `2^9`:

```
9 -> 4 -> 2 -> 1 -> 0
```

Fast power makes fewer calls because it cuts the exponent almost in half each time.

10.9 Time and Space Complexity of Fast Power

Time Complexity

```
O(log n)
```

Why?

The exponent is divided by `2` again and again.

Example:

```
n = 16
16 -> 8 -> 4 -> 2 -> 1 -> 0
```

This is logarithmic growth.

Space Complexity

```
O(log n)
```

Why?

The recursion depth is about the number of times we divide `n` by `2`.

So the stack depth is `O(log n)`.

---

11. Basic Power vs Fast Power

Function	Example	Answer	Time Complexity	Space Complexity
`power(2, 9)`	`2^9`	`512`	`O(n)`	`O(n)`
`fastPower(2, 9)`	`2^9`	`512`	`O(log n)`	`O(log n)`

Both give the same answer.

But fast power reaches the answer using fewer recursive calls.

---

12. Full Week 4 Practice Program

```c
#include <stdio.h>

int sum(int n) {
    if (n == 0) {
        return 0;
    }

    return sum(n - 1) + n;
}

int factorial(int n) {
    if (n < 0) {
        return -1;
    }

    if (n == 0) {
        return 1;
    }

    return factorial(n - 1) * n;
}
```

```c
int power(int m, int n) {
    if (n == 0) {
        return 1;
    }

    return power(m, n - 1) * m;
}

int fastPower(int m, int n) {
    if (n == 0) {
        return 1;
    }

    if (n % 2 == 0) {
        return fastPower(m * m, n / 2);
    }

    return m * fastPower(m * m, (n - 1) / 2);
}

int main() {
    printf("sum(5) = %d\n", sum(5));
    printf("factorial(5) = %d\n", factorial(5));
    printf("power(2, 9) = %d\n", power(2, 9));
    printf("fastPower(2, 9) = %d\n", fastPower(2, 9));

    return 0;
}
```

12.1 Expected Output

```
sum(5) = 15
factorial(5) = 120
power(2, 9) = 512
fastPower(2, 9) = 512
```

---

13. Main Pattern in Week 4

All Week 4 recursive functions follow the same basic structure.

```
return smaller_problem + or * current_work;
```

Examples:

```
return sum(n - 1) + n;
```

```
return factorial(n - 1) * n;
```

```
return power(m, n - 1) * m;
```

The function keeps reducing the input until it reaches the base case.

Then it solves everything while returning.

---

14. Summary Table

Problem	Base Case	Recursive Case	Time	Space
Sum	`sum(0) = 0`	`sum(n) = sum(n - 1) + n`	`O(n)`	`O(n)`
Factorial	`factorial(0) = 1`	`factorial(n) = factorial(n - 1) * n`	`O(n)`	`O(n)`
Basic Power	`power(m, 0) = 1`	`power(m, n) = power(m, n - 1) * m`	`O(n)`	`O(n)`
Fast Power	`fastPower(m, 0) = 1`	Cut exponent in half	`O(log n)`	`O(log n)`

---

15. Common Beginner Mistakes in Week 4

Mistake 1: Forgetting the Base Case

Wrong:

```
int sum(int n) {
    return sum(n - 1) + n;
}
```

Problem:

```
There is no stopping condition.
```

Correct:

```
int sum(int n) {
    if (n == 0) {
        return 0;
    }

    return sum(n - 1) + n;
}
```

Mistake 2: Not Moving Toward the Base Case

Wrong:

```
int sum(int n) {
    if (n == 0) {
        return 0;
    }

    return sum(n) + n;
}
```

Problem:

```
n never changes.
```

So the function keeps calling itself with the same input.

Correct:

```
return sum(n - 1) + n;
```

Mistake 3: Using the Wrong Base Case for Factorial

Wrong:

```
if (n == 0) {
    return 0;
}
```

Problem:

```
factorial(5) would become 0 because multiplication by 0 destroys the result.
```

Correct:

```
if (n == 0) {
    return 1;
}
```

Mistake 4: Ignoring Negative Input in Factorial

Wrong:

```
factorial(-5)
```

without checking `n < 0`.

Problem:

```
The function moves away from 0 and may never stop.
```

Correct:

```
if (n < 0) {
    return -1;
}
```

Mistake 5: Thinking Fast Power and Basic Power Have the Same Speed

They do not.

Basic power reduces by `1`:

```
9 -> 8 -> 7 -> 6 -> 5 -> 4 -> 3 -> 2 -> 1 -> 0
```

Fast power reduces by about half:

```
9 -> 4 -> 2 -> 1 -> 0
```

So fast power is much better for large exponents.

---

16. How to Trace Week 4 Problems

Use this method every time:

1. Write the function call.
2. Identify the base case.
3. Identify the recursive case.
4. Write the calling phase.
5. Stop when the base case is reached.
6. Write the returning phase.
7. Calculate the final answer.
8. Count the number of calls.
9. Find time complexity.
10. Find space complexity.

Example for `factorial(4)`:

Calling Phase

```
factorial(4) waits for factorial(3) * 4
factorial(3) waits for factorial(2) * 3
factorial(2) waits for factorial(1) * 2
factorial(1) waits for factorial(0) * 1
factorial(0) returns 1
```

Returning Phase

```
factorial(1) = 1 * 1 = 1
factorial(2) = 1 * 2 = 2
factorial(3) = 2 * 3 = 6
factorial(4) = 6 * 4 = 24
```

Final answer:

```
24
```

---

17. Week 4 Practice Questions

Question 1

What is the output?

```
printf("%d", sum(4));
```

Answer:

```
10
```

Reason:

```
1 + 2 + 3 + 4 = 10
```

Question 2

What is the output?

```
printf("%d", factorial(4));
```

Answer:

```
24
```

Reason:

```
4 * 3 * 2 * 1 = 24
```

Question 3

What is the output?

```
printf("%d", power(3, 4));
```

Answer:

```
81
```

Reason:

```
3^4 = 3 * 3 * 3 * 3 = 81
```

Question 4

What is the base case of recursive sum?

Answer:

```
sum(0) = 0
```

Question 5

What is the base case of factorial?

Answer:

```
factorial(0) = 1
```

Question 6

Why does recursive power have `O(n)` time complexity?

Answer:

```
Because the exponent decreases by 1 each time, so the function makes about n
recursive calls.
```

Question 7

Why does fast power have `O(log n)` time complexity?

Answer:

```
Because the exponent is divided by 2 again and again.
```

---

18. Week 4 Mastery Checklist

Before moving to Week 5, you should be able to do these:

- Explain the recursive formula for sum.
- Trace `sum(5)` manually.
- Explain why sum performs addition during the returning phase.

- Write recursive sum in C.
- Write loop-based sum in C.
- Explain the recursive formula for factorial.
- Explain why `factorial(0)` is `1`.
- Handle negative factorial input safely.
- Trace `factorial(5)` manually.
- Write recursive factorial in C.
- Write loop-based factorial in C.
- Explain the recursive formula for basic power.
- Trace `power(2, 5)` manually.
- Write recursive power in C.
- Explain why basic power is `O(n)`.
- Explain the idea behind fast power.
- Explain the difference between even and odd exponent handling.
- Trace `fastPower(2, 9)` manually.
- Explain why fast power is `O(log n)`.
- Compare recursive and loop solutions.
- Explain why recursion usually uses more memory than loops.

---

19. Week 4 Final Summary

Week 4 teaches basic recursive problem solving.

The main problems are:

```
sum(n)
factorial(n)
power(m, n)
fastPower(m, n)
```

The main pattern is:

```
Big problem = smaller problem + current work
```

For sum:

```
sum(n) = sum(n - 1) + n
```

For factorial:

```
factorial(n) = factorial(n - 1) * n
```

For power:

```
power(m, n) = power(m, n - 1) * m
```

For fast power:

```
Reduce the exponent by half instead of reducing it by one.
```

Simple recursive sum, factorial, and basic power usually have:

```
Time complexity: O(n)
Space complexity: O(n)
```

Fast power has:

```
Time complexity: O(log n)
Space complexity: O(log n)
```

The most important Week 4 lesson is:

> Do not only write recursive code. Trace it, understand its base case, understand its returning phase, and compare its memory usage with a loop.

Week 1 taught what recursion is.

Week 2 taught how recursion uses memory.

Week 3 taught how to classify recursion.

Week 4 teaches how to solve basic recursive problems.

---

20. Final Rule for Week 4

Every time you write a recursive mathematical function, ask these questions:

1. What is the smallest valid input?
2. What should the function return at that smallest input?

3. How can I reduce the current problem?
4. What work happens after the smaller problem returns?
5. Does the input always move toward the base case?
6. What is the time complexity?
7. What is the space complexity?
8. Would a loop use less memory?

If you can answer these questions, you are not memorizing recursion. You are understanding it.